# Stochastic Control

Homework Project 1

Reinforcement Learning with Policy Gradient

Ioannis Dimoulios

Mat. Num. 470364

Winter Semester 2024

**Problem 1**

a) The state space is $\mathbb{X}$ is defined by the set

$$\mathbb{X} = \{(y, z) | \, y \in \{1, \dots, 5\}, \, z \in \{1, \dots, 10\}\}.$$

b) The action space can be defined as follows

$$\mathbb{U} = \{s, \nu, d\},$$

where, assuming $(y, z)$ is the current state of the the robot,

- $s$ stands for *same*, i.e. the robot stays on the same $y$ and moves only on the $z$ direction with $z \leq 9$,
- $\nu$ stands for *up*, i.e. the robot increments both its $y$ and $z$ coordinates with $z \leq 9$ and $y \leq 4$,
- $d$ stands for *down*, i.e. the robot decrements its $y$ coordinate and increments the $z$ coordinate with $z \leq 9$ and $y \geq 2$.

c) Let

$$\mathcal{B} = \{-r_{\text{obst}} \, , -r_{\text{diag}}, r_{\text{end}} \},$$

where $r_{\text{end}} \gg r_{\text{obst}} > r_{\text{diag}}$. The reward set $\mathcal{R}$ can be defined as follows

$$\mathcal{R} = \left\{ \sum_{x \in \mathcal{A}} x \, \middle| \, \mathcal{A} \subseteq \mathcal{B} \right\}.$$

Then, $\mathcal{R}$ contains the sums of the elements of each subset of $\mathcal{B}$. Note that $0 \in \mathcal{R}$, as the sum of the elements of the empty subset. In this setting award

- $-r_{\text{obst}}$ when the robot steps on an obstacle cell,
- $-r_{\text{diag}}$ when the robot makes a diagonal movement, i.e. the action $u$ is $\nu$ or $d$,
- $r_{\text{end}}$ when the robot reaches a desired end position.

If more than one of the above conditions apply for a movement of the robot, then award the sum of the corresponding values. If none apply, then award $0$.

d) Let $x = (y, z)$ be the current state and $x^+ = (y', z + 1)$ the next state. Then, once the action $u$ is known both the state transition and the reward are deterministic, i.e.

$$\mathbf{q}(\{x^+, r\}; x, u) = 1,$$

where $r \in \mathcal{R}$ is calculated according to the rules outlined in (c). For example, if $u \in \{\nu, d\}$ and $x^+$ is a desired end position, then $r = r_{\text{end}} - r_{\text{diag}}$, and if $u = s$ and $x^+$ is an obstacle-free non-end position, then $r = 0$.
For the state transitions, if

- $u = s$, then $y' = y$ with $z \leq 9$,
- $u = \nu$, then $y' = y + 1$ with $z \leq 9$ and $y \leq 4$,
- $u = d$, then $y' = y - 1$ with $z \leq 9$ and $y \geq 2$.

Note that if $z = 10$, the robot has reached an end position and the path ends. Also note that illegal actions ($u = \nu$ and $u = d$ when $y = 5$ and $y = 1$ respectively) have been taken into account by forcing $\pi(u; x) = 0$.

**Problem 2**

I am seriously conflicted as to what I'm supposed to write here.
a) The SPG is essentially a special case of the basic PG with $N = 1$. Indeed, basic PG tries out different trajectories per iteration of $M$ and averages them to get the gradient of the objective function $J(\theta)$ and subsequently a better parametrization $\theta$ through gradient ascent. The time complexity is $\mathcal{O}(MN)$.
On the contrary, SPG samples one trajectory per update step leading to a more computationally efficient algorithm and computes the next step of the parametrization $\theta$ accordingly. Of course, this means that the algorithm may produce inconsistent results if the variance gradient of the objective functions is unbounded. The time complexity is $\mathcal{O}(M)$. Hence, SPG maybe used when time and computational resources are limited.
SPG could also be useful when the actual global optimal parametrization lies very far away from the initial one and hence the updates need to be more drastic in order to avoid other local optima. In this scenario, PG would not be suitalbe given its more conservative update loop.

b) The motivation to employ the SPG instead of the PG stems from the fact that as long as $\theta$ updates to the direction of individual paths which on average produce the real gradient of the objective function, then it should work. Indeed, we view $\theta_k$ where $k = 1, \dots, M$ as a series of random variables in the same way as section **3.1** of the SGD notes.
Now, let $\omega = (x_0, u_0, \dots, x_T, r_T)$ be a path generated by the policy $\pi^{\theta_k}$. Then, in similar fashion to the notes, we may write

$$\theta_{k+1} = \theta_k + \eta \nabla_\theta J_\omega(\theta_k),$$

where

$$\nabla_\theta J_\omega(\theta_i) = \sum_{k=1}^{T} R_k \sum_{\ell=0}^{T-1} \nabla_\theta \log \pi^{\theta_i}(U_\ell; X_\ell).$$

But then

$$\mathbf{E}_\omega \left[\nabla_\theta J_\omega(\theta)\right] = \nabla_\theta J(\theta)$$

which makes the estimator of the gradient unbiased as desired. (I have barely convinced myself for all the above...)

c) The 'may be used' part should be answered in part (a). For convergence guarantee, we need the following:

- unbiased estimation, which is self-fulfilled based on (b),
- bounded variance of $\nabla_\theta J_\omega(\theta)$, so that it doesn't diverge or oscillate, and
- a step size $\eta$ which is neither too small so that the algorithm can make meaningful updates in terms size during the first iterations and nor too big so that it doesn't overtake optimal solutions. In particular, a decaying step size would be ideal.

**Problem 3**

First, we will outline some implementation details. The simulations were done in Julia. Initially, the gradient of the softmax function is given by the following. Let $u_1 = s, u_2 = \nu, u_3 = d$ and $\pi^\theta(u_i; s) = \dfrac{e^{\theta(s,u_i)}}{\sum_{v \in \mathbb{U}} e^{\theta(s,v)}}$. Then

$$\nabla_\theta \log \pi^\theta(u_i; x) = -\begin{bmatrix}\pi^\theta(u_1; x) & \pi^\theta(u_2; s) & \pi^\theta(u_3; s)\end{bmatrix}^T + e_i,$$

where $e_i$ is the standard basis vector at the i-th direction. This makes the calculation of $\nabla_\theta J_\omega(\theta_i)$ straightforward.

Then, the way the policy gradient algorithms (both PG and SPG) are implemented in the pseudocode makes it particularly hard to define policy convergence in a consistent and convenient way. Indeed, the standard definition where

$$\left|\pi^{\theta_{i+1}}(u;x) - \pi^{\theta_i}(u;x)\right| < \epsilon$$

for all $(x,u)$ is never satisfied for small enough $\epsilon$, as every time the initial state $x_0$ changes a spike is produced in the sequence of norms of differences of policies. This is prevalent in Figure 1. The same happens for the differences of parametrizations $\theta$ instead of policies. The other option would be convergence in the sum of rewards, but since each initial value has vastly different optimal trajectories and hence rewards, this didn't work either.

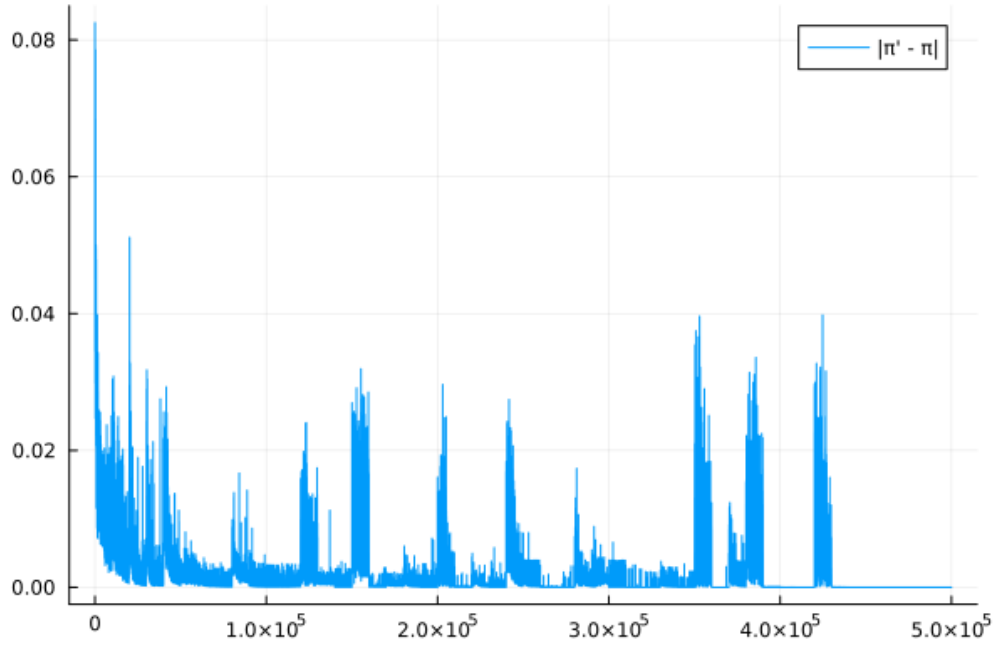



Figure 1: Euclidean norm of the difference of vector policies vs number of episodes for the PG algorithm with $M = 10^3$.

As far as the value of the constant step, we believe the most consistent one was in the range $\eta \in (0.001, 0.003)$.
The constants $r_{\text{obst}}, r_{\text{diag}}, r_{\text{end}}$ where chosen to be $10, 6, 50$ respectively, so that the robot always looked for the desired end positions and two diagonal movements were worse than an obstacle.
The initial parametrization $\theta_0$ has $\theta_0(x,u) = 1$ as long as $u$ is an allowed action on state $x$.
And then we had my 7 year old mid-range laptop fighting against our unoptimized software implementation and crashing every other simulation or taking 30 minutes to complete it when $M = 10^5$ or refusing to draw some of the graphs when the array length was too big.

a) Since we didn't manage to produce any convincing graphs, we will claim that when SPG converged, it was faster than PG, where faster means it needed less episodes to reach a (sub)optimal policy.
We could not complete any simulations for PG with $N \geq 100$, but intuitively the bigger $N$ gets the more slowly PG converges, but the convergence becomes less noisy. Indeed, the more episodes per update the more $g_T/N$ approaches the theoretical value of the gradient and hence the update becomes more accurate.

b) The optimal policy was found was found using SPG with $M = 10^5, \eta = 0.003$ and by traversing the initial states on the outer loop of the algorithm in reverse order, i.e. by decrementing $z$.
The bulk of simulations showed that once the robot gets to a state with $z \geq 5$ it picks the optimal actions. However, the policy generated by various simulations struggled to find the "correct" actions when $z$ was smaller especially when $r_{\text{obst}} \approx 2r_{\text{diag}}$ which is the case in our setting.
Another observation is that when the robot starts from a cell which does not belong to the optimal paths of starting states with smaller $z$ it may make a suboptimal action, which is to be expected as essentially it is less trained to move starting

from that state. These kind of errors are minimized when paths are allowed to start from an obstacle position during the simulation.
In terms of actual total reward, the optimal policy has an average total reward having an episode generated for every allowed (obstacle-free) position of $R_{\text{opt, avg}} = 37.45$. In Figure 2, it is apparent that this policy performs better than other generated policies by SPG with fewer training episodes.

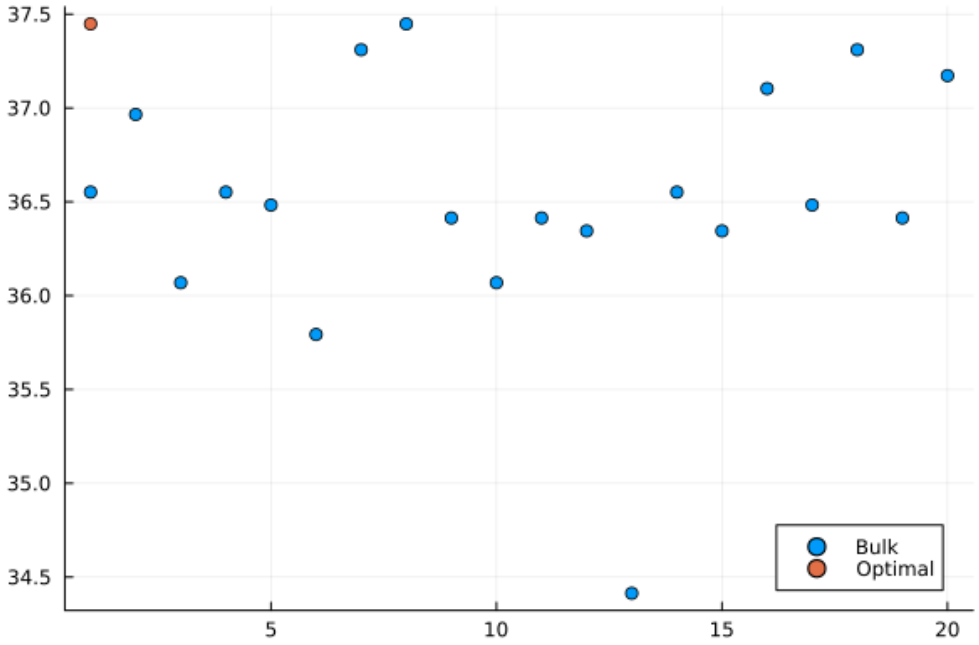
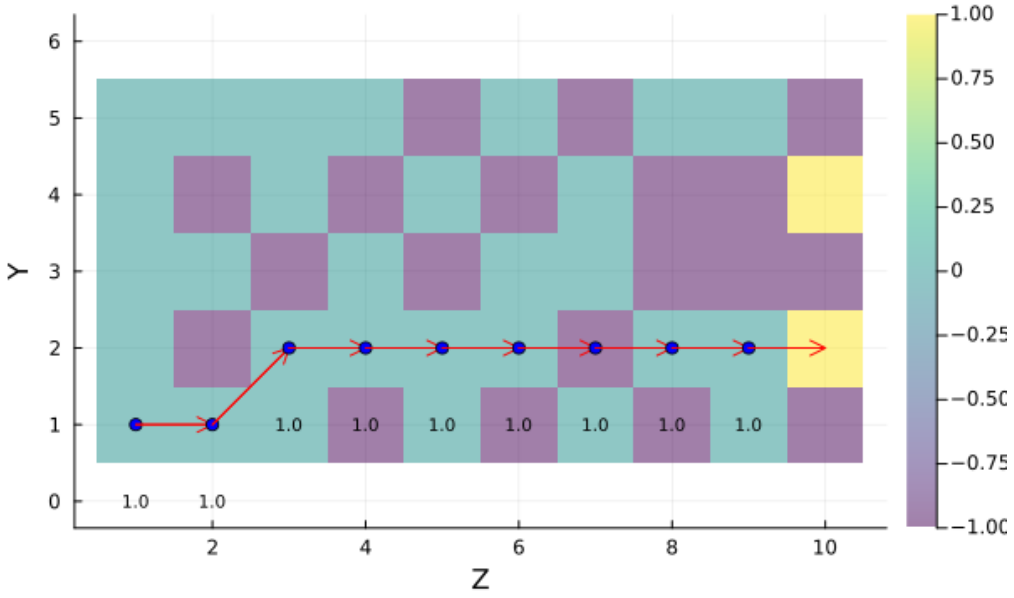
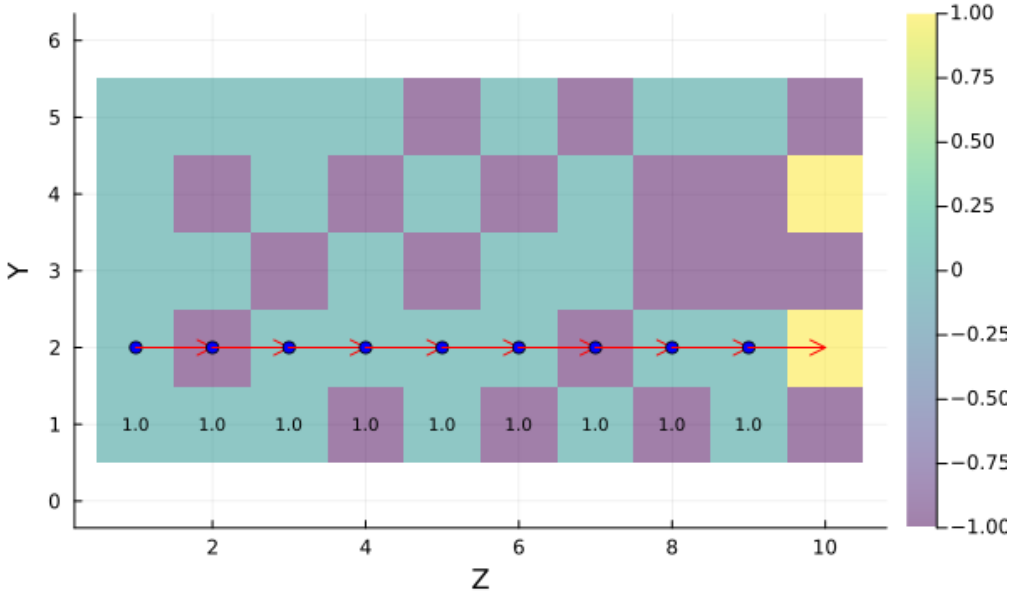
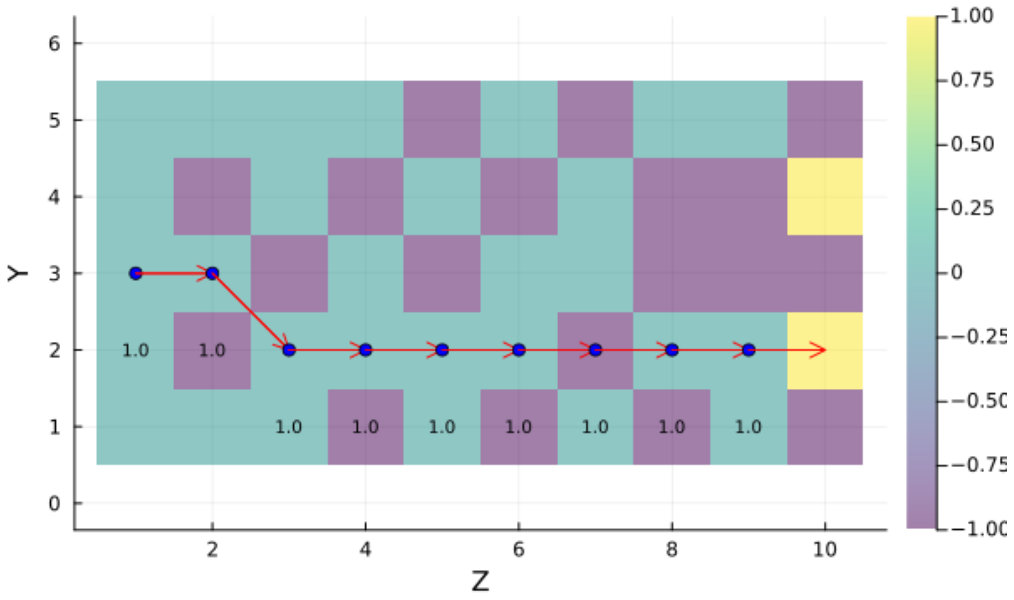


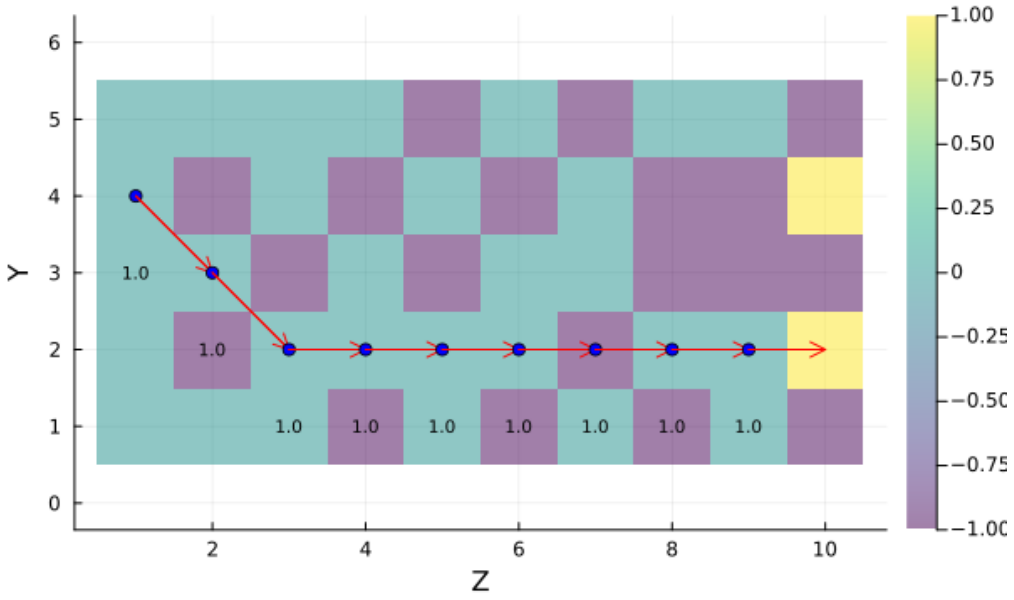

Figure 2: Average episode total reward by averaging over one generated episode for each legal starting position
(Blue) Bulk generated policies by SPG with $M = 10^4$ vs (Orange) Optimal policy found by SPG with $M = 10^5$

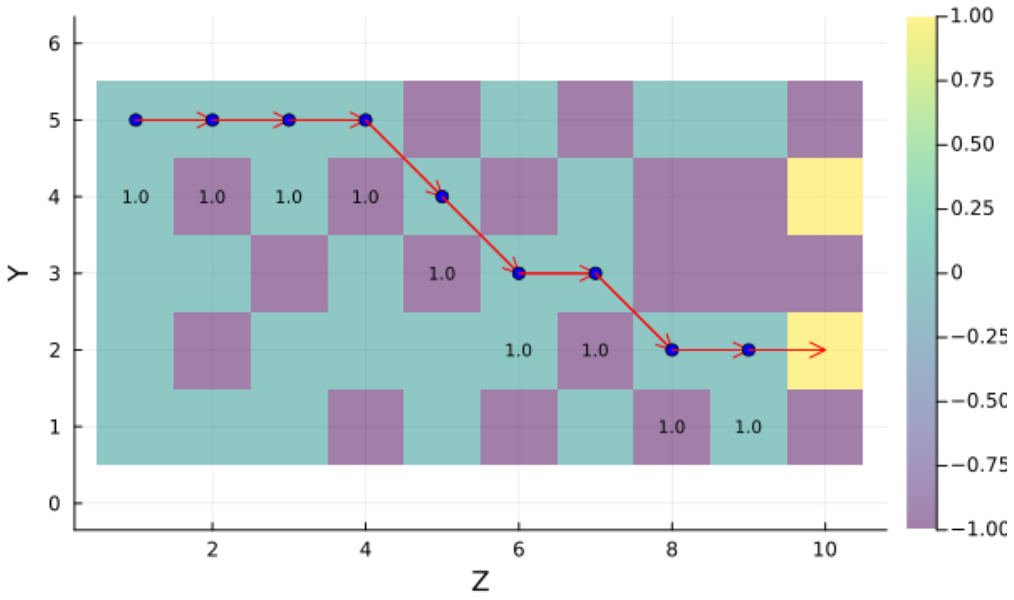
Figures in the next pages.
c) This optimal control policy is essentially deterministic, i.e. for each state one particular action is chosen with probability 1.0 rounding to the second decimal digit. The probabilities of the actions in the particular paths presented below are shown right below the arrows. The blue cells are obstacle free, the purple are obstacles and the yellow ones are the desired-end cells.
We are going to claim boldly that this policy is desirable as for each starting position it generates the maximal total reward. It is worth-noting though that the stochastic element of the process almost vanishes. As a result other optimal paths are completely eradicated, e.g. on the episode starting from $(1, 1)$ another optimal path would move straight to $(1, 3)$ and move up to match the designated one.
Also, notice how the robot steps on an obstacle when this decision saves 2 diagonal moves.

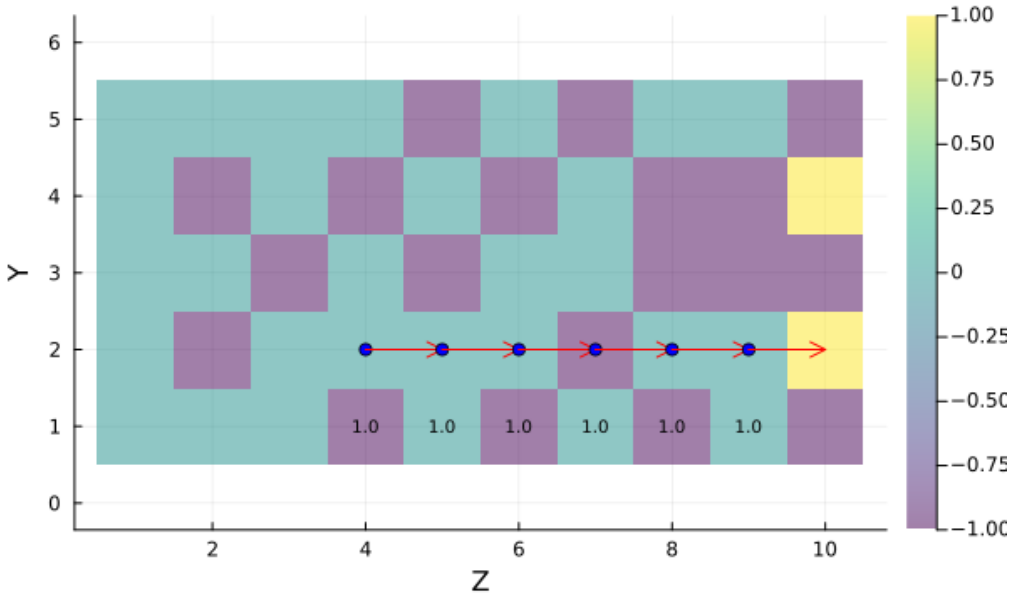

Y
Z
1.0
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00

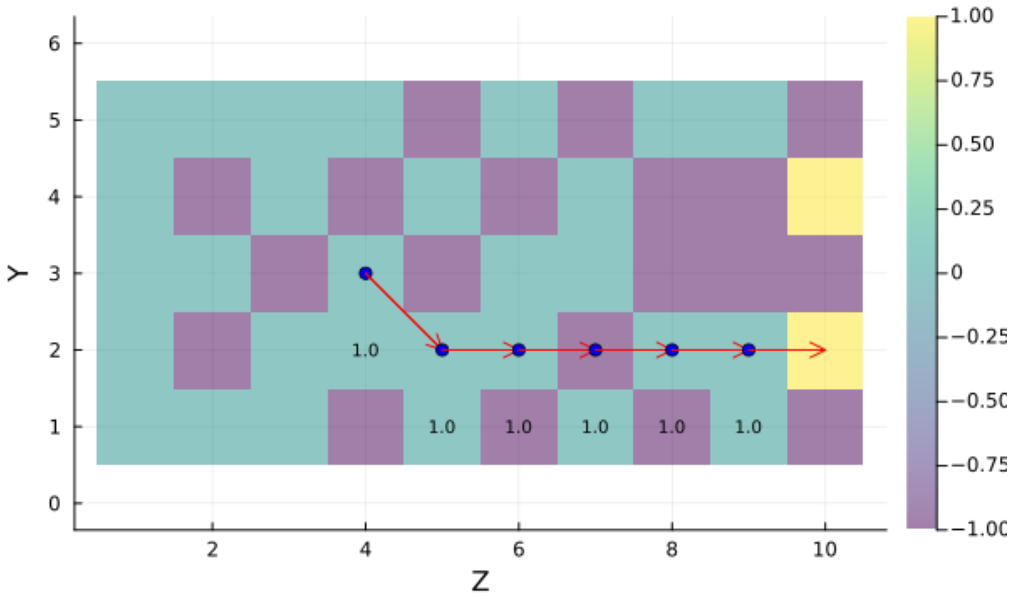

Y
Z
1.0
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00

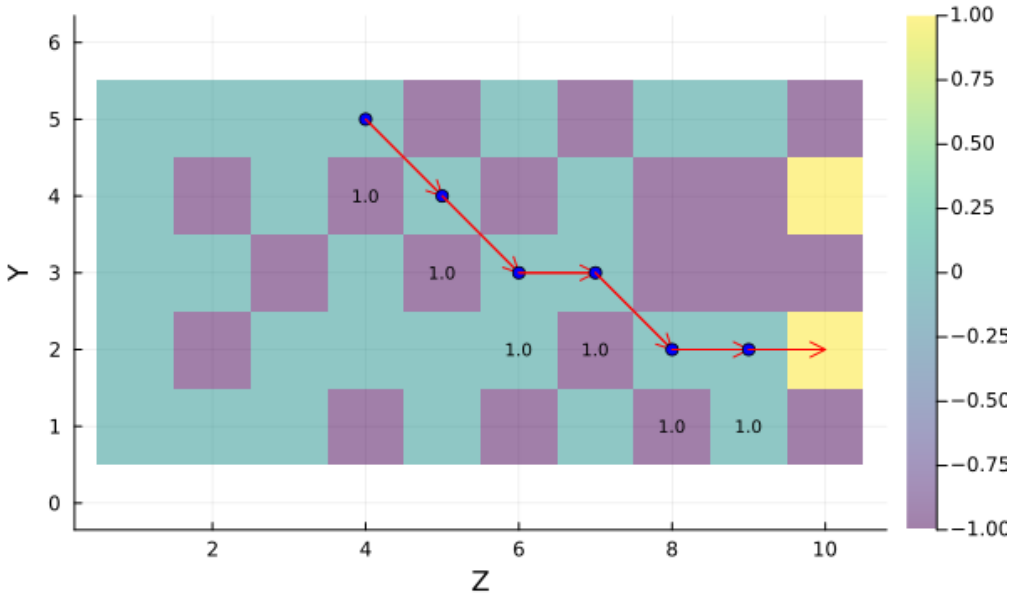

Y
Z
1.0
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00


Y
Z
1.0
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00


1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
Y
Z
0
1
2
3
4
5
6
2
4
6
8
10
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00


1.0
1.0
1.0
1.0
1.0
1.0
Y
Z
0
1
2
3
4
5
6
2
4
6
8
10
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00


Y
Z
1.0
1.0
1.0
1.0
1.0
1.0
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00


Y
Z
1.0
1.0
1.0
1.0
1.0
1.00
0.75
0.50
0.25
0
−0.25
−0.50
−0.75
−1.00